

CSE 565 Structural-Based Testing Project

Coleman Chase

August 29, 2026

1 Introduction

This report presents the results of a structural-based testing assignment. Part 1 uses test cases for the provided `VendingMachine.java` program and discusses statement and decision coverage. Part 2 uses PMD to perform static source code analysis on `StaticAnalysis.java` and evaluates the anomalies detected by the tool.

2 Part 1: Vending Machine Testing

2.1 Tool Used and Coverage Type

For Part 1, a Java test harness was used to execute the test cases for `VendingMachine.java`. The harness calls the `dispenseItem` method with selected inputs and compares each actual output to the expected output.

The goal of the test suite is to exercise statement coverage and decision coverage. Statement coverage measures whether executable statements are run by the tests. Decision coverage measures whether control-flow decisions, such as `if` and `else if` conditions, are evaluated through their relevant outcomes.

2.2 Test Cases Developed

The test cases were selected to cover the available products, successful purchases, exact purchases, and insufficient-money cases. Table 1 summarizes the cases.

Table 1: Vending Machine Test Cases

Test	Input	Item	Expected Result
Candy with extra money	30	candy	Item dispensed and change of 10 returned
Coke exact money	25	coke	Item dispensed.
Coffee exact money	45	coffee	Item dispensed.
Coffee, enough for candy or coke	30	coffee	Item not dispensed, missing 15 cents. Can purchase candy or coke.

Test	Input	Item	Expected Result
Coffee, enough for candy only	22	coffee	Item not dispensed, missing 23 cents. Can purchase candy.
Coffee, not enough for anything	10	coffee	Item not dispensed, missing 35 cents. Cannot purchase item.

2.3 Test Harness Code

The following code was written to execute the test cases.

Listing 1: Vending Machine Test Harness

```
public class VendingMachineTestHarness {
    private static int testsRun = 0;
    private static int testsPassed = 0;

    public static void main(String[] args) {
        testDispenseCandyWithChange();
        testDispenseCokeExactChange();
        testDispenseCoffeeExactChange();
        testMissingCoffeeCanPurchaseCandyOrCoke();
        testMissingCoffeeCanPurchaseCandy();
        testMissingCoffeeCannotPurchaseItem();

        System.out.println("Passed " + testsPassed + " of " + testsRun + " tests.
    ");
    }

    private static void testDispenseCandyWithChange() {
        assertEquals(
            "candy with extra money returns item and change",
            "Item dispensed and change of 10 returned",
            VendingMachine.dispenseItem(30, "candy")
        );
    }

    private static void testDispenseCokeExactChange() {
        assertEquals(
            "coke exact money returns item",
            "Item dispensed.",
            VendingMachine.dispenseItem(25, "coke")
        );
    }

    private static void testDispenseCoffeeExactChange() {
        assertEquals(
            "coffee exact money returns item",
```

```

        "Item dispensed.",
        VendingMachine.dispenseItem(45, "coffee")
    );
}

private static void testMissingCoffeeCanPurchaseCandyOrCoke() {
    assertEquals(
        "coffee with 30 cents reports candy or coke alternative",
        "Item not dispensed, missing 15 cents. Can purchase candy or coke
.",
        VendingMachine.dispenseItem(30, "coffee")
    );
}

private static void testMissingCoffeeCanPurchaseCandy() {
    assertEquals(
        "coffee with 22 cents reports candy alternative",
        "Item not dispensed, missing 23 cents. Can purchase candy.",
        VendingMachine.dispenseItem(22, "coffee")
    );
}

private static void testMissingCoffeeCannotPurchaseItem() {
    assertEquals(
        "coffee with 10 cents reports no affordable item",
        "Item not dispensed, missing 35 cents. Cannot purchase item.",
        VendingMachine.dispenseItem(10, "coffee")
    );
}

private static void assertEquals(String testName, String expected, String
actual) {
    testsRun++;
    if (expected.equals(actual)) {
        testsPassed++;
        System.out.println("PASS: " + testName);
    } else {
        System.out.println("FAIL: " + testName);
        System.out.println("  expected: " + expected);
        System.out.println("  actual:   " + actual);
    }
}
}

```

2.4 Test Execution Screenshot

Figure 1 shows the test harness output. All six test cases passed.

```
ASU > cd /Users/chase/Desktop/code/ASU ; /usr/bin/env /Library/Java/JavaVirtualMachines/temurin-8.jdk/Contents/Home/bin/java -cp /Users/chase/Library/Application\ Support/Code/User/workspaceStorage/1dd763ce9f22b9022f5ae9e8f496c5c8/redhat.java/jdt_ws/ASU_d54217ab/bin VendingMachineTestHarness
PASS: candy with extra money returns item and change
PASS: coke exact money returns item
PASS: coffee exact money returns item
PASS: coffee with 30 cents reports candy or coke alternative
PASS: coffee with 22 cents reports candy alternative
PASS: coffee with 10 cents reports no affordable item
Passed 6 of 6 tests.
ASU >
```

Figure 1: Vending machine test execution output

2.5 Coverage Discussion

The test suite was designed to exercise the main control-flow paths in `dispenseItem`. The first three tests cover the product selection statements for candy, coke, and coffee. The successful-purchase tests cover the `input > cost` and `input == cost` branches. The remaining three tests cover the insufficient-money branch and the nested decisions for `input < 45`, `input < 25`, and `input < 20`.

Together, these tests execute the statements that assign product costs, compute change, assign successful return messages, assign insufficient-money messages, and return the final result. The insufficient-money tests were intentionally chosen at 30, 22, and 10 cents so that the nested decision logic is exercised across its different outcomes.

One implementation issue is that the vending machine code compares strings using `==`. These tests use Java string literals such as `"candy"`, `"coke"`, and `"coffee"`, so the current implementation behaves as expected because string literals are interned. In production code, `.equals()` would be safer because `==` compares object references rather than string contents.

3 Part 2: Static Analysis

3.1 Tool Used and Analysis Type

PMD 6.55.0 was used as the static source code analysis tool. PMD analyzes source code without executing the program. It can detect unused variables, questionable object comparisons, style issues, design issues, empty code blocks, duplicated code, overly complex methods, and other maintainability problems.

PMD was downloaded as a standalone distribution and run from the command line. The command used was:

Listing 2: PMD Command

```
/tmp/pmd-bin-6.55.0/bin/run.sh pmd -d StaticAnalysis.java -R rulesets/java/quickstart.xml -f text > pmd_static_analysis_report.txt
```

3.2 PMD Report

The PMD report generated for `StaticAnalysis.java` is shown below.

Listing 3: PMD Static Analysis Report

```
StaticAnalysis.java:4: NoPackage: All classes, interfaces, enums and
    annotations must belong to a named package
StaticAnalysis.java:5: UseUtilityClass: All methods are static. Consider
    using a utility class instead. Alternatively, you could add a private
    constructor or make the class abstract to silence this warning.
StaticAnalysis.java:15: UnusedLocalVariable: Avoid unused local variables such
    as 'weight'.
StaticAnalysis.java:16: UnusedLocalVariable: Avoid unused local variables such
    as 'length'.
StaticAnalysis.java:24: CompareObjectsWithEquals: Use equals() to compare
    object references.
StaticAnalysis.java:24: UseEqualsToCompareStrings: Use equals() to compare
    strings instead of '==' or '!='
StaticAnalysis.java:34: ControlStatementBraces: This statement should have braces
StaticAnalysis.java:36: ControlStatementBraces: This statement should have braces
```

3.3 Anomalies Detected

PMD detected unused local variables in the constructor on lines 15 and 16:

```
int weight = 0;
String length = "";
```

These variables are assigned values but are never used. This is a data flow anomaly because the variables are defined without any later use.

PMD also detected a string comparison problem on line 24:

```
if(product == "Electronics")
```

This is risky because Java's `==` operator compares object references. String contents should normally be compared with `.equals()`, such as:

```
if ("Electronics".equals(product))
```

PMD also reported additional style and design findings, including no package declaration, a utility-class recommendation, and missing braces around control statements. These findings are useful, but the unused-variable and string-comparison findings are the most relevant anomalies for this assignment.

3.4 Assessment of PMD

PMD was effective for this assignment because it identified the major suspicious patterns in `StaticAnalysis.java`. The report included the line number, rule name, and a short description for each finding, which made the results easy to interpret.

The main features of PMD include command-line analysis, configurable rulesets, multiple output formats, and Java quality rules for correctness, maintainability, and style. PMD covers anomalies such as unused variables, questionable comparisons, empty statements, unnecessary code, and design problems.

The tool was easy to use because it required only a source file, a ruleset, and an output format. The main limitation is that PMD reports both important anomalies and minor style issues, so the user must interpret which findings are most relevant to the assignment.

4 Conclusion

The vending machine test cases exercise the main statement and decision paths in the provided program. The PMD static analysis identified the important anomalies in `StaticAnalysis.java`, especially unused variables and unsafe string comparison. Together, the two parts demonstrate both dynamic testing through executed test cases and static analysis through source code inspection.